**UNIVERSIDAD DE ANTIOQUIA**

**FACULTAD DE INGENIERÍA**

**PROGRAMA DE INGENIERÍA DE TELECOMUNICACIONES**

**SEMESTRE II**

# Desafío 2

*Amigos para siempre, con Ude@Book*

Paradigma programación orientada a objetos – POO

**Preinforme**

**Asignatura:**
Informática II Teoría

**Autores:**
Yudis Paola Apreciado Jiménez
Oscar Miguel Lopez Peñata

**Profesor:**
Augusto Salazar Jiménez

Medellín, Octubre de 2026

## 1. Comprensión del desafío

El problema plantea el desarrollo de la **capa lógica de una red social** denominada **Ude@Book**. Se tiene como restricción el uso de la librería estándar de plantillas (STL), estructuras tipo `struct` o librerías de terceros para la solución del ejercicio. Todo el núcleo del programa debe abarcar la programación orientada a objetos, empleando gestión manual de memoria dinámica.

## 2. Decisiones para el diseño

En este desafío utilizaremos módulos independientes divididos en archivos de cabecera (`.h`) e implementación (`.cpp`), el cual llevará las siguientes clases:

- **Nodo T:** almacenará un dato genérico (`T`) y tendrá dos punteros (`siguiente` y `anterior`), el cual se definirá de forma doblemente enlazada para facilitar recorridos bidireccionales necesarios en la visualización de fotos y formato carrusel.
- **Lista ligada T:** con esta reemplazamos los contenedores de la STL. La utilizamos para la inserción al final, eliminación de nodos y la sobrecarga del operador `[]`, permitiendo interactuar con la lista como si fuera un arreglo estático, pero manteniendo la flexibilidad de la memoria dinámica.
- **Usuario:** es la clase central del proyecto. Almacenará información de perfil mediante punteros a carácter `char*` para gestionar cadenas dinámicas sin recurrir a `std::string`. También va a contener instancias de la clase lista ligada para administrar de forma separada las relaciones con otros usuarios, las solicitudes pendientes, álbumes creados y mensajes enviados y recibidos.
- **Mensajes:** esta clase estará destinada para almacenar el cuerpo de texto, la marca temporal de envío (fecha/hora) y el estado de lectura. Es necesario que contenga punteros para objetos emisor y receptor (`Usuario*`) en lugar de almacenar copias por valor, para cumplir con la indicación de tener eficiencia en la memoria.
- **Álbum y Foto:** estas clases representan la jerarquía de almacenamiento en la plataforma. Vamos a modelarlas bajo la relación de composición; el ciclo de vida de la fotografía depende del álbum. En la clase álbum definimos el nivel de privacidad y tendrá la lista de fotos asociadas, mientras que Foto será para almacenar las rutas

absolutas del archivo en formato Linux y el contador de iteraciones (like).

- **Plataforma:** esta clase la definimos para que actúe como clase controladora del sistema, es decir, se encargará de persistir el estado global: lista de todos los usuarios registrados en el sistema. También centrará las funciones de datos: carga y actualización de archivos planos, y centrará los acumuladores globales encargados de la medición del consumo de recursos.

Para este desafío consideraremos la eficiencia y el manejo de memoria.

- **Paso por referencia:** en las operaciones de inserción y consulta en lista y métodos inter-clase se evitará el paso de objetos por valor. Priorizaremos el uso de referencias constantes, por ejemplo: `const Usuario&` o punteros directos `Usuario*`. De esta forma mitigamos la sobrecarga en la pila y evitamos duplicaciones innecesarias de bytes en el heap.
- **Medición de memoria:** para cumplir con lo solicitado en el punto VII, implementaremos un registro interno dentro de la clase `Plataforma` que mapeará el tamaño de bytes de las estructuras creadas. Cada asignación donde el operador `new` intervenga incrementará el contador según el `sizeof` de la estructura, y con `delete` lo decrementará. De esta forma garantizamos un cálculo transparente y en tiempo real.

## 3. Diagrama UML

En la Figura 1 se presenta el diagrama de clases de la solución propuesta. El diseño consta de **siete clases** organizadas en tres zonas lógicas:

- **Zona de control:** `Plataforma`, clase controladora que administra la lista global de usuarios, la persistencia en archivos planos y las métricas de consumo de recursos.
- **Zona de dominio:** `Usuario`, `Mensaje`, `Album` y `Foto`. `Usuario` es la clase central, con listas independientes para amigos, solicitudes, mensajes y álbumes. `Mensaje` contiene punteros a emisor y receptor. `Album` y `Foto` forman una jerarquía de composición.
- **Zona de estructuras genéricas:** `ListaLigada<T>` y `Nodo<T>`, plantillas que reemplazan los contenedores de la STL y se instancian para `Usuario*`, `Mensaje*`, `Album*` y `Foto*`.

Las relaciones del diagrama incluyen **composición** (rombo negro) entre `Plataforma-Usuario`,

`Usuario-Album`, `Album-Foto` y `ListaLigada-Nodo`; **asociación** entre `Mensaje` y `Usuario` (roles emisor y receptor); y dos **autorelaciones** en `Usuario` (`esAmigoDe` y `solicita`). Las cardinalidades (`1`, `0..N`) reflejan la multiplicidad de cada relación.

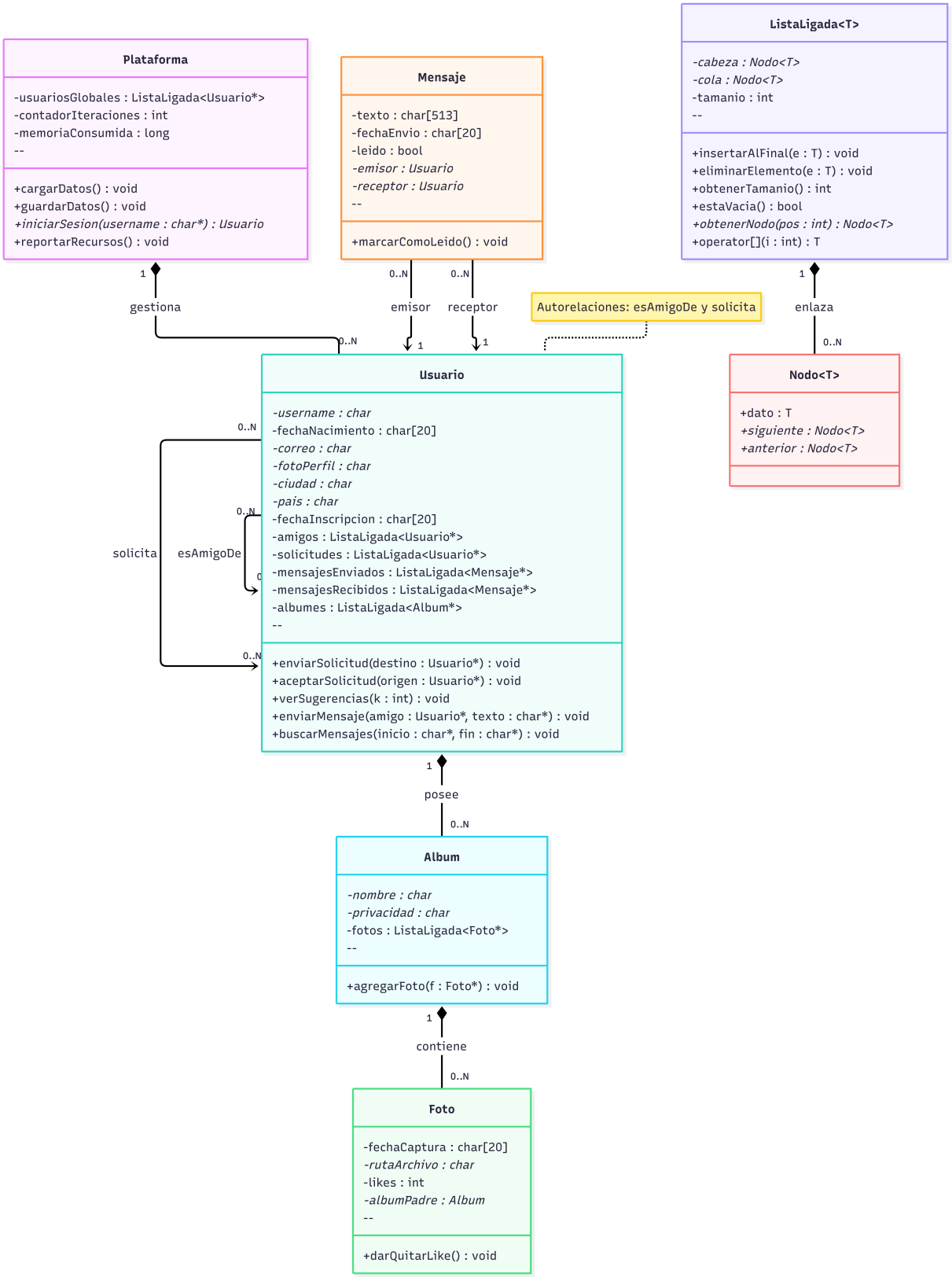



**Figura 1** Diagrama UML